

# Adaptive DSR Route Cache

Parameter sweep comparing mobility-scaled LinkCache timers against fixed DSR cache behavior

## Thesis

In high mobility, DSR should trust cached routes for less time. The experiment tests whether speed-scaled cache timers improve delivery.

50

NODES

Random waypoint MANET

10

FLOWES

one-to-one UDP pairs

4

SWEEP POINTS

10%, 50%, 110%, 150%

200s

RUNTIME

one CSV row per second

**DSR cache reuse is valuable until mobility turns cached routes stale.**

## Why the cache can hurt

- DSR avoids repeated route discovery by keeping learned source routes and link state.
- In RandomWaypoint mobility, a route that worked earlier can point through a now-broken link.
- Long cache lifetimes can keep stale state alive long enough to drop packets before repair.

## Adapt cache trust to movement

The experiment keeps the default MANET scenario and metrics, but computes cache timers from node speed before installing DSR.

**Higher node speed -> shorter route and link stability timers**

A single mobility scale converts base cache timers into effective DSR values.

$$\text{adaptive\_value} = \text{clamp}(\text{base\_value} * \text{cache\_reference\_speed} / \text{node\_speed}, \text{min\_value}, \text{max\_value})$$

### Formula variables

base\_value: original timer multiplied by the sweep percentage  
cache\_reference\_speed: speed where base timers are preserved, 5 m/s  
node\_speed: scenario speed used for cache trust, 20 m/s  
clamp: keeps timers inside configured safety bounds

### This sweep used a 0.25 mobility scale

$\text{cache\_reference\_speed} / \text{node\_speed} = 5 / 20 = 0.25$

Example: the 50% timeout starts at 150 s, then becomes  $\text{clamp}(150 * 0.25, 30, 300) = 37.5$  s.

**30-300s**

ROUTE CACHETIMEOUT  
route lifetime bounds

**2-25s**

INIT STABILITY  
new link confidence

**10-120s**

USE EXTENDS  
successful-use extension

The adaptive values are computed before DSR is installed on the nodes.

## Command-line switches

adaptiveRouteCache  
adaptiveLinkStability  
cacheType  
baseRouteCacheTimeout  
baseInitStability  
baseUseExtends  
cacheReferenceSpeed

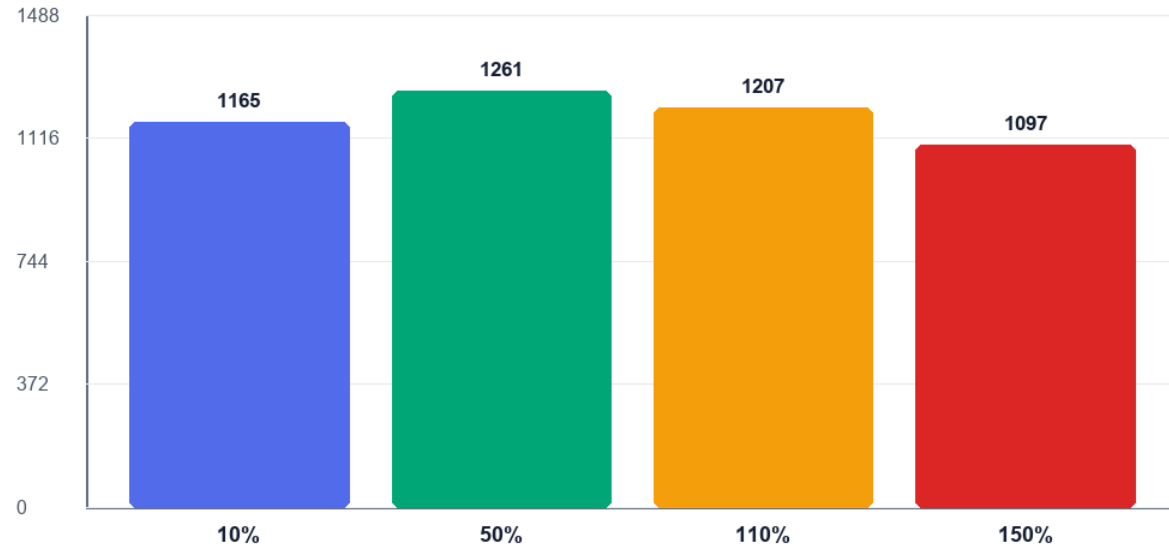
```
Config::SetDefault("ns3::dsr::DsrRouting::Route  
CacheTimeout",  
  
TimeValue(Seconds(m_effectiveRouteCacheTimeout)  
));  
Config::SetDefault("ns3::dsr::DsrRouting::InitS  
tability",  
  
TimeValue(Seconds(m_effectiveInitStability)));  
Config::SetDefault("ns3::dsr::DsrRouting::UseEx  
tends",  
  
TimeValue(Seconds(m_effectiveUseExtends)));
```

**Placement matters: defaults must be set before  
dsrMain.Install(...) creates the routing objects.**

The 50% adaptive setting delivered the strongest packet count in this run.

### Adaptive sweep: total packets

Packets received over 200 seconds



**1261**

BEST PACKET COUNT

50% adaptive run

**3.228**

AVG KBPS

full 200-second average

**37.5s**

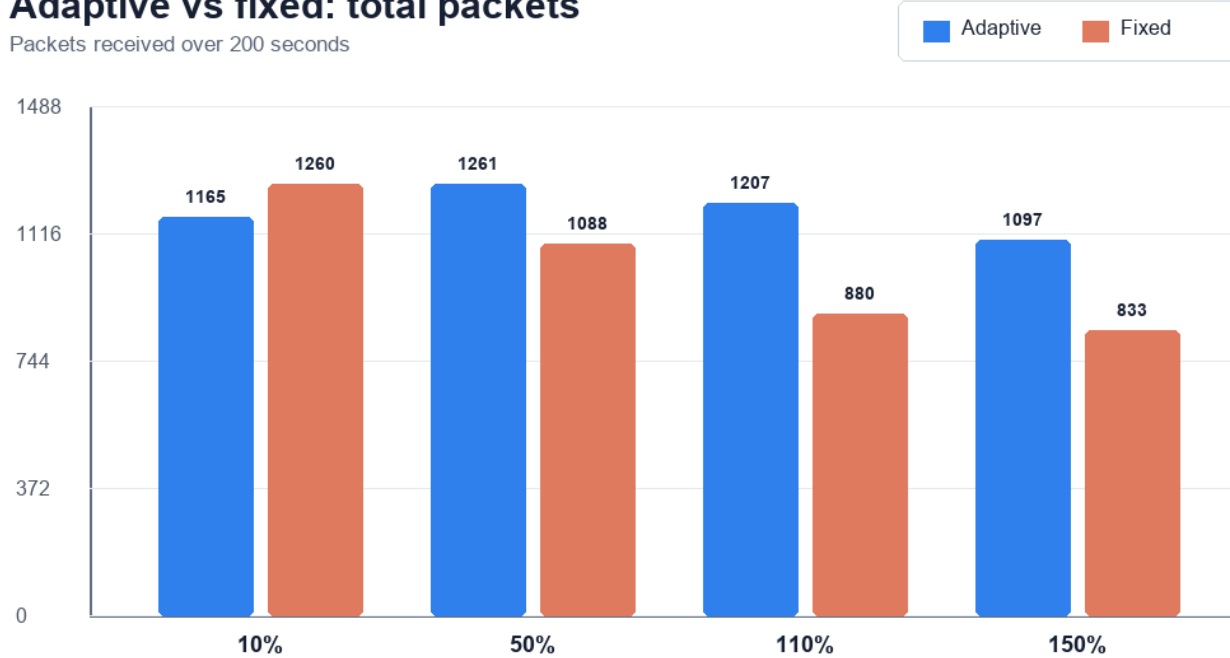
EFFECTIVE TIMEOUT

50% base after scaling

## Adaptive tuning beat fixed cache behavior at 50%, 110%, and 150%.

### Adaptive vs fixed: total packets

Packets received over 200 seconds



**+173**

50% PACKET DELTA  
adaptive vs fixed

**+327**

110% PACKET DELTA  
largest improvement

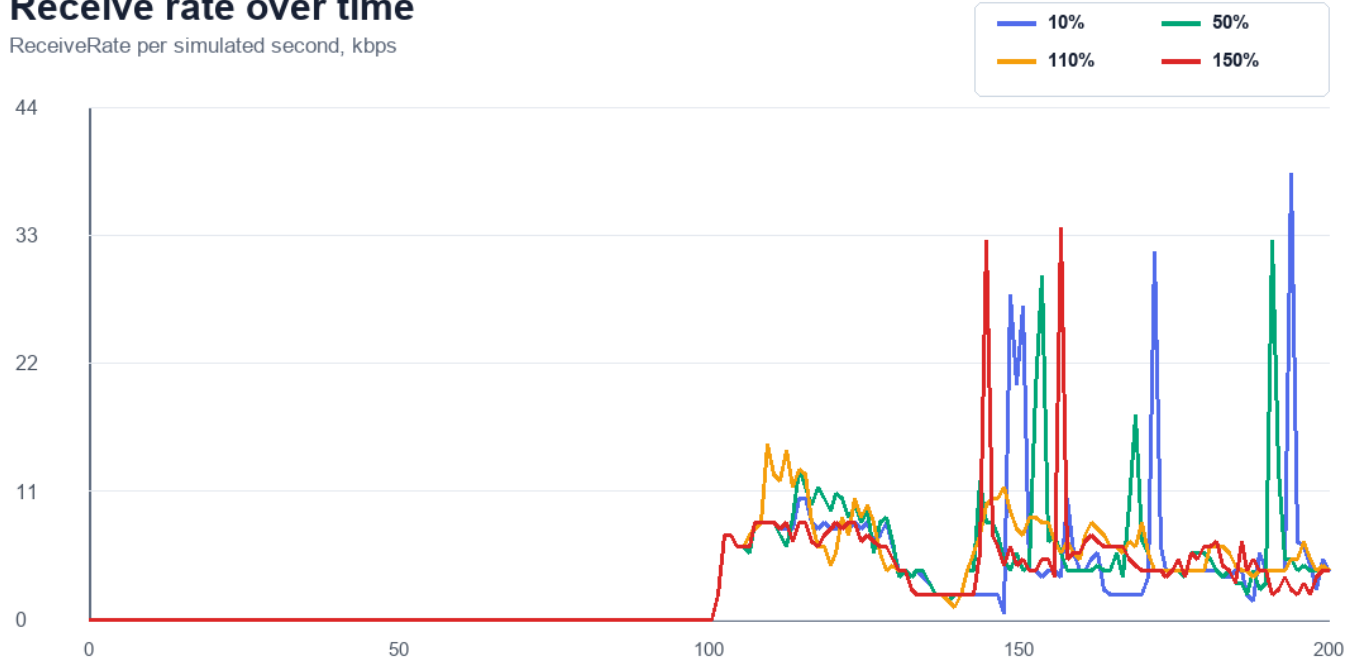
**-95**

10% PACKET DELTA  
low-end clamp loses

Packet delivery starts after traffic begins and differs mainly during active seconds.

## Receive rate over time

ReceiveRate per simulated second, kbps



## How to read it

Rows are one simulated second. Traffic starts around 100 s, so the first half of each metrics file is intentionally zero. The comparisons use total packets and average kbps across the full 200 s for consistency with the default ns-3 example.

## Adaptive cache timing looks promising, but it needs seed and scenario sweeps next.

### What the experiment suggests

- A moderate effective timeout outperformed both very short and long-lived cache assumptions.
- The adaptive policy helped most when fixed DSR would otherwise keep stale cache state too long.
- The 10% case exposes the impact of clamp bounds, so low-end tuning deserves separate attention.

### Future work

Repeat runs across random seeds  
Sweep cacheReferenceSpeed  
independently  
Compare LinkCache and PathCache  
Sweep node speed and node density  
Track routing overhead alongside delivery

## 3/4

ADAPTIVE WINS  
against fixed comparator

## 50%

BEST SWEEP POINT  
1261 packets received

## next

VARIANCE TESTING  
multi-seed validation